

به نام خدا

نام درس: علوم داده

نام استاد: دکتر نادری

گزارش: HW4

نام و نام خانوادگی: نیوشا یقینی

شماره دانشجویی: ۹۸۵۲۳۳۴۶

تاریخ: دی/۱۴۰۳

۱. Regularization چطور کار می‌کند؟ چگونه باعث جلوگیری از Overfitting می‌شود؟

Regularization یک تکنیک برای کاهش پیچیدگی مدل و جلوگیری از Overfitting است. Overfitting زمانی رخ می‌دهد که مدل بیش از حد به داده‌های آموزش وابسته شود و نویز داده‌ها را نیز بیاموزد.

Regularization با اضافه کردن یک جریمه به تابع هزینه، مقادیر وزن‌های مدل را محدود می‌کند و از رشد بیش از حد آن‌ها (Overfitting) جلوگیری می‌کند. این جریمه باعث ساده‌تر شدن مدل و کاهش اثر ویژگی‌های کم‌اهمیت یا نویزی می‌شود.

انواع Regularization:

- L1 (Lasso): وزن برخی ویژگی‌ها را صفر می‌کند و برای انتخاب ویژگی‌ها مناسب است.
- L2 (Ridge): وزن‌ها را کوچک می‌کند ولی به صفر نمی‌رساند، برای کنترل پیچیدگی مدل استفاده می‌شود.
- Elastic Net: ترکیبی از L1 و L2 است.

نتیجه: Regularization با ساده کردن مدل و کاهش اثر نویز، تعمیم‌پذیری را بهبود می‌بخشد و از یادگیری بیش از حد جزئیات غیرضروری جلوگیری می‌کند.

۲. کدام نوع Regularization برای Feature Selection مناسب است؟ L1 یا L2؟ چگونه؟

Feature Selection چیست؟

Feature Selection فرآیندی است که طی آن ویژگی‌های مهم و مرتبط انتخاب و ویژگی‌های غیرضروری یا نویزی حذف می‌شوند. این کار باعث ساده‌تر شدن مدل، افزایش دقت و کاهش Overfitting می‌شود.

مزایای Feature Selection

۱. کاهش Overfitting با حذف نویز.
۲. افزایش سرعت پردازش مدل.
۳. ساده‌تر و قابل تفسیرتر شدن مدل.
۴. بهبود دقت با حذف ویژگی‌های غیرمرتبط.

کدام نوع Regularization برای Feature Selection مناسب است؟ L1 یا L2؟

L1 Regularization (Lasso) برای Feature Selection مناسب‌تر است زیرا وزن برخی ویژگی‌های غیرضروری را به صفر می‌رساند و آن‌ها را از مدل حذف می‌کند. این ویژگی باعث می‌شود که مدل فقط با ویژگی‌های مهم و تاثیرگذار کار کند.

L2 Regularization (Ridge) وزن ویژگی‌ها را کوچک می‌کند ولی هیچ‌کدام را حذف نمی‌کند. بنابراین، برای Feature Selection مناسب نیست.

اگر هدف انتخاب ویژگی‌های مهم باشد، **L1 Regularization** گزینه بهتری است زیرا به‌طور موثر ویژگی‌های کم‌اهمیت یا نویزی را حذف می‌کند.

۳. تفاوت Domain Adaptation و Transfer Learning چیست؟ مقایسه کنید.

Domain Adaptation و Transfer Learning هر دو زیرمجموعه‌ای از یادگیری انتقالی (Transfer Learning) هستند و هدف مشترک آن‌ها بهبود عملکرد مدل در یک دامنه هدف (Target Domain) با استفاده از دانش کسب‌شده از یک دامنه منبع (Source Domain) است. با این حال، تفاوت‌های کلیدی در رویکرد و هدف آن‌ها وجود دارد:

تعریف و هدف

Transfer Learning:

- **تعریف:** انتقال دانش از یک دامنه منبع به یک دامنه هدف، حتی اگر این دو دامنه متفاوت باشند. تفاوت می‌تواند در داده‌ها، ویژگی‌ها، یا وظایف باشد.
- **هدف:** بهبود عملکرد در دامنه هدف با استفاده از دانش عمومی یا قبلی، بدون توجه به اینکه دامنه منبع و دامنه هدف کاملاً مشابه یا متفاوت باشند.
- **مثال:** استفاده از مدل از پیش آموزش‌دیده‌ای مانند ResNet (آموزش‌دیده روی ImageNet) برای یک مجموعه داده جدید (مانند طبقه‌بندی بیماری‌ها از تصاویر پزشکی).

Domain Adaptation:

- **تعریف:** انتقال دانش بین دو دامنه (منبع و هدف) که وظیفه یکسانی دارند اما توزیع داده‌هایشان متفاوت است.
- **هدف:** کاهش اثر تغییر توزیع داده (Covariate Shift) بین دامنه‌ها و اطمینان از عملکرد بهتر مدل در دامنه هدف.
- **مثال:** استفاده از یک مدل آموزش‌دیده روی داده‌های نوشته دست‌خط انگلیسی (منبع) برای پیش‌بینی روی داده‌های نوشته دست‌خط فارسی (هدف).

تفاوت‌ها

ویژگی	Transfer Learning	Domain Adaptation
ارتباط دامنه‌ها	دامنه منبع و هدف می‌توانند کاملاً متفاوت باشند.	دامنه منبع و هدف باید وظیفه مشابهی داشته باشند.
هدف وظیفه	وظیفه دامنه منبع و هدف می‌تواند متفاوت باشد.	وظیفه دامنه منبع و هدف یکسان است.
تفاوت توزیع داده‌ها	توزیع داده‌ها ممکن است مشابه یا متفاوت باشد.	تأکید بر تفاوت توزیع داده‌ها و کاهش آن است.
مثال مدل‌ها	مدل‌های عمومی مانند BERT یا ResNet.	تکنیک‌های سازگار مانند Adversarial Domain Adaptation یا Fine-tuning.
پیچیدگی انتقال	ممکن است نیاز به تطبیق ویژگی‌ها نداشته باشد.	معمولاً نیاز به تطبیق ویژگی‌ها یا بازنمایی‌ها دارد.

- هر دو به دنبال استفاده از دانش موجود برای بهبود عملکرد در شرایط جدید هستند.
- در هر دو، مدل از دانش دامنه منبع بهره می‌گیرد تا کارایی بهتری در دامنه هدف داشته باشد.

➤ Transfer Learning:

- یادگیری مدل‌های عمومی با استفاده از داده‌های عظیم (مثلاً GPT برای زبان)
- کاربرد در مسائل کوچک با داده‌های محدود.

➤ Domain Adaptation:

- یادگیری روی داده‌هایی که برچسب‌گذاری آن‌ها دشوار یا پرهزینه است.
- کاربرد در پردازش زبان طبیعی (مثلاً تطبیق مدل‌های زبانی برای لهجه‌های مختلف).

۴. چطور K را در الگوریتم K -means انتخاب می‌کنیم؟ چگونه ممکن است در این الگوریتم $Overfitting$ رخ دهد؟ توضیح دهید.

چگونه K را در الگوریتم K -means انتخاب می‌کنیم؟

الگوریتم K -means نیاز به تعداد خوشه‌ها (K) به عنوان یک ورودی دارد. انتخاب مقدار مناسب برای K اهمیت زیادی دارد. روش‌های رایج برای انتخاب مقدار K عبارتند از:

۱. روش Elbow:

- این روش از تابع هزینه الگوریتم K -means به نام $WCSS$ (Within-Cluster Sum of Squares) استفاده می‌کند.
- $WCSS$ مجموع مربعات فاصله نقاط از مراکز خوشه‌ها است. با افزایش K ، $WCSS$ کاهش می‌یابد.
- نمودار $WCSS$ در برابر مقادیر مختلف K رسم می‌شود. نقطه‌ای که کاهش $WCSS$ به تدریج متوقف می‌شود (شکل آرنج یا Elbow، مقدار مناسب K را نشان می‌دهد).

۲. روش Silhouette Coefficient:

- این روش کیفیت خوشه‌بندی را با مقایسه نزدیکی هر نقطه به خوشه خودش و سایر خوشه‌ها اندازه‌گیری می‌کند.
- مقادیر بین -1 تا 1 است و مقدار نزدیک به 1 نشان‌دهنده خوشه‌بندی مناسب است K . را طوری انتخاب می‌کنیم که این معیار حداکثر شود.

۳. روش Gap Statistics:

- این روش تفاوت بین $WCSS$ واقعی داده‌ها و $WCSS$ داده‌های تصادفی را بررسی می‌کند. مقدار K را طوری انتخاب می‌کنیم که این اختلاف بیشینه باشد.

چگونه ممکن است *Overfitting* در *K-means* رخ دهد؟

Overfitting در *K-means* به معنای خوشه‌بندی بیش از حد دقیق داده‌ها است که می‌تواند باعث کاهش تعمیم‌پذیری شود. این معمولاً در شرایط زیر رخ می‌دهد:

۱. انتخاب مقدار بسیار بزرگ برای K :
 - اگر K بیش از حد بزرگ باشد، هر نقطه می‌تواند به یک خوشه جداگانه اختصاص یابد. این وضعیت باعث می‌شود مدل به داده‌های آموزشی وابسته شود و نتواند الگوهای کلی داده‌ها را تشخیص دهد.
۲. داده‌های نویزی:
 - اگر داده‌ها شامل نویز باشند، الگوریتم ممکن است این نویز را نیز به خوشه‌های جداگانه اختصاص دهد، که می‌تواند به *Overfitting* منجر شود.
۳. توزیع نابرابر داده‌ها:
 - اگر داده‌ها به طور نابرابر توزیع شده باشند، *K-means* ممکن است خوشه‌هایی ایجاد کند که نماینده واقعی داده‌ها نیستند.

راه‌های جلوگیری از *Overfitting*:

- استفاده از روش‌های مناسب برای انتخاب K (مثل Elbow یا Silhouette)
- پیش‌پردازش داده‌ها و حذف نویز.
- تنظیم خوشه‌ها با داده‌های جدید و بررسی تعمیم‌پذیری.

نتیجه: انتخاب مقدار مناسب K برای جلوگیری از *Overfitting* بسیار مهم است. با استفاده از روش‌هایی مثل Elbow و بررسی کیفیت خوشه‌بندی می‌توان مقدار K بهینه را تعیین کرد. همچنین باید داده‌ها را پیش‌پردازش کرد تا از تأثیر نویز جلوگیری شود.

۵. **Bias-Variance Tradeoff** را ابتدا به صورت ساده توضیح دهید. سپس یک نمونه از این مقایسه با کمک MSE , MAE برای بررسی مدل بنویسید و محدودیت‌های هر **Metric** را بررسی کنید.

در یادگیری ماشین، هدف یافتن تعادل بین (*Bias* خطای سیستماتیک) و (*Variance* واریانس) است. این دو مفهوم به این صورت تعریف می‌شوند:

۱. **Bias** (خطا ناشی از ساده بودن مدل):
 - زمانی رخ می‌دهد که مدل بیش از حد ساده باشد و نتواند الگوهای داده را به درستی یاد بگیرد. (*Underfitting*)
 - نتیجه: پیش‌بینی‌های ضعیف روی داده‌های آموزشی و تست.
۲. **Variance** (خطا ناشی از پیچیدگی مدل):
 - زمانی رخ می‌دهد که مدل بیش از حد پیچیده باشد و جزئیات و نویز داده‌های آموزشی را یاد بگیرد. (*Overfitting*)
 - نتیجه: عملکرد خوب روی داده‌های آموزشی اما ضعیف روی داده‌های جدید.

Tradeoff

یک مدل خوب باید تعادلی بین *Bias* و *Variance* برقرار کند. افزایش پیچیدگی مدل می‌تواند *Bias* را کاهش دهد اما ممکن است *Variance* را افزایش دهد، و برعکس.

مثال با MSE و MAE

فرض کنید یک مدل رگرسیون خطی داریم و می‌خواهیم عملکرد آن را روی داده‌ها بررسی کنیم.

۱. Mean Squared Error (MSE):

- تابع هزینه‌ای که میانگین مربع خطاها را محاسبه می‌کند.
- نسبت به مقادیر بزرگ خطا حساس است، زیرا خطاها را به توان دو می‌برد.
- محدودیت: در صورت وجود Outlier (نقاط پرت)، MSE مقدار زیادی خواهد داشت.

۲. Mean Absolute Error (MAE):

- میانگین قدر مطلق خطاها را محاسبه می‌کند.
- نسبت به Outlier حساسیت کمتری دارد.
- محدودیت: نمی‌تواند به خوبی خطاهای کوچک را نشان دهد.

مقایسه مدل‌ها:

فرض کنید دو مدل داریم:

- مدل ۱: خطای آموزش پایین (Bias کم)، خطای تست بالا (Variance بالا).
- مدل ۲: خطای آموزش و تست متوسط (تعادل بین Bias و Variance)

برای ارزیابی:

- اگر مدل مستعد Outlier باشد، MAE مناسب‌تر است.
- اگر بخواهیم به خطاهای بزرگ وزن بیشتری بدهیم، MSE را انتخاب می‌کنیم.

نتیجه‌گیری:

Bias-Variance Tradeoff نشان‌دهنده اهمیت تعادل بین ساده و پیچیده بودن مدل است. با استفاده از متریک‌هایی مثل MSE و MAE می‌توان عملکرد مدل را بررسی کرد، اما باید محدودیت‌های هر متریک در نظر گرفته شود.

۶. فرض کنید یک دیتاست داریم و می‌خواهیم یک مدل ماشین لرنینگ روی آن آموزش بدهیم. آیا برای افزایش دقت می‌توانیم پیچیدگی مدل را هر چقدر خواستیم زیاد کنیم؟ برای دستیابی به حداکثر دقت باید به چه عواملی توجه کنیم؟ توضیح دهید.

آیا برای افزایش دقت می‌توانیم پیچیدگی مدل را هر چقدر خواستیم زیاد کنیم؟

خیر، افزایش پیچیدگی مدل به طور نامحدود باعث بهبود دقت نمی‌شود. پیچیدگی بیش از حد مدل می‌تواند منجر به Overfitting شود، جایی که مدل داده‌های آموزشی را به خوبی پیش‌بینی می‌کند، اما در داده‌های جدید (داده‌های تست) عملکرد ضعیفی دارد. از طرف دیگر، یک مدل بیش از حد ساده ممکن است به Underfitting منجر شود، جایی که مدل حتی روی داده‌های آموزشی نیز عملکرد ضعیفی دارد.

برای دستیابی به حداکثر دقت باید به چه عواملی توجه کنیم؟

۱. تعادل بین Bias و Variance:
 - مدل نباید خیلی ساده (Bias بالا) یا خیلی پیچیده (Variance بالا) باشد. تعادل بین این دو برای دستیابی به بهترین تعمیم‌پذیری ضروری است.
۲. کیفیت داده‌ها:
 - داده‌های تمیز و بدون نویز باعث می‌شود مدل بهتر آموزش ببیند.
 - داده‌ها باید دارای ویژگی‌های معنادار و مرتبط با خروجی باشند.
۳. حجم داده‌ها:
 - مدل‌های پیچیده به داده‌های بیشتری برای یادگیری نیاز دارند. در غیر این صورت، ممکن است Overfitting رخ دهد.
۴. انتخاب ویژگی‌ها (Feature Selection):
 - انتخاب ویژگی‌های مفید و حذف ویژگی‌های کم‌اهمیت یا نویزی می‌تواند به بهبود عملکرد مدل کمک کند.
۵. تنظیم هایپرپارامترها:
 - با استفاده از تکنیک‌هایی مثل Grid Search یا Random Search می‌توان مقادیر بهینه برای هایپرپارامترها (مانند تعداد لایه‌ها یا نرخ یادگیری) را پیدا کرد.
۶. Regularization:
 - افزودن جریمه به تابع هزینه (مانند L1 یا L2 Regularization) می‌تواند از Overfitting جلوگیری کند و دقت مدل را بهبود دهد.
۷. Cross-Validation:
 - با استفاده از روش‌های اعتبارسنجی (مثل K-fold cross-validation)، می‌توان عملکرد مدل را روی داده‌های مختلف بررسی کرد تا از تعمیم‌پذیری آن اطمینان حاصل شود.
۸. انتخاب مدل مناسب:
 - نوع مسئله (طبقه‌بندی، رگرسیون، یا خوشه‌بندی) و پیچیدگی داده‌ها تعیین‌کننده انتخاب مدل مناسب است. گاهی مدل ساده‌تر (مثل Logistic Regression) بهتر از مدل پیچیده‌تر (مثل Neural Networks) عمل می‌کند.

نتیجه‌گیری:

برای افزایش دقت، نمی‌توان تنها با افزایش پیچیدگی مدل به هدف رسید. باید به عواملی مثل کیفیت داده‌ها، تنظیم هایپرپارامترها، استفاده از Regularization و انتخاب مدل مناسب توجه کرد. هدف اصلی ایجاد مدلی است که هم روی داده‌های آموزشی و هم روی داده‌های آزمون عملکرد خوبی داشته باشد.

۷. Precision/Recall tradeoff را با یک مثال توضیح دهید و Confusion matrix مربوطه را بنویسید.

Precision/Recall Tradeoff چیست؟

Precision و Recall دو معیار مهم در ارزیابی مدل‌های طبقه‌بندی هستند. معمولاً بین این دو معیار یک نوع تضاد (Tradeoff) وجود دارد، به این معنا که افزایش یکی ممکن است باعث کاهش دیگری شود. این تضاد زمانی بیشتر آشکار می‌شود که شما تصمیم‌گیری مدل را با تغییر آستانه (Threshold) کنترل کنید.

تعریف *Precision* و *Recall*:

- **Precision**: درصد نمونه‌هایی که مدل به‌درستی به‌عنوان مثبت پیش‌بینی کرده است، از میان همه نمونه‌هایی که به‌عنوان مثبت پیش‌بینی شده‌اند.

$$Precision = \frac{TP}{TP + FP}$$

- **Recall**: درصد نمونه‌های مثبت واقعی که مدل آن‌ها را به‌درستی تشخیص داده است.

$$Recall = \frac{TP}{TP + FN}$$

مثال:

فرض کنید یک مدل تشخیص بیماری دارید که افراد بیمار (کلاس مثبت) را از افراد سالم (کلاس منفی) تفکیک می‌کند. خروجی مدل یک احتمال است، و با تغییر آستانه تصمیم‌گیری می‌توانید روی *Precision* و *Recall* تأثیر بگذارید:

۱. **Threshold بالا** (افزایش *Precision*):

- مدل فقط افرادی را مثبت پیش‌بینی می‌کند که از نظر آن کاملاً مطمئن است.
- **نتیجه**: تعداد (*FP* مثبت‌های اشتباه) کاهش می‌یابد، ولی ممکن است برخی *TP*ها (مثبت‌های واقعی) را از دست بدهید. (کاهش *Recall*)

۲. **Threshold پایین** (افزایش *Recall*):

- مدل حساس‌تر می‌شود و تعداد بیشتری از افراد را به‌عنوان مثبت پیش‌بینی می‌کند.
- **نتیجه**: تعداد *FN* (مثبت‌های از دست رفته) کاهش می‌یابد، ولی ممکن است تعداد *FP* افزایش یابد (کاهش *Precision*).

Confusion Matrix:

فرض کنید داده‌های شما شامل ۱۰۰ نمونه است و نتایج پیش‌بینی مدل به شرح زیر است:

	پیش‌بینی مثبت	پیش‌بینی منفی
واقعی مثبت	40 (TP)	10 (FN)
واقعی منفی	15 (FP)	35 (TN)

Precision و *Recall* محاسبه می‌شوند:

$$Precision = \frac{TP}{TP + FP} = \frac{40}{40 + 15} = 0.727$$

$$Recall = \frac{TP}{TP + FN} = \frac{40}{40 + 10} = 0.8$$

تضاد Precision و Recall

- اگر بخواهید Recall را به ۱ برسانید، باید همه نمونه‌ها را به‌عنوان مثبت پیش‌بینی کنید. این کار Precision را کاهش می‌دهد زیرا تعداد FP افزایش می‌یابد.
- اگر بخواهید Precision را به ۱ برسانید، باید فقط نمونه‌هایی که مدل ۱۰۰٪ مطمئن است را مثبت پیش‌بینی کنید. این کار Recall را کاهش می‌دهد زیرا بسیاری از TP ها به FN تبدیل می‌شوند.

نتیجه‌گیری:

Precision و Recall باید بر اساس نیاز مسئله متعادل شوند. مثلاً در تشخیص بیماری، Recall مهم‌تر است (تشخیص ندادن بیماری می‌تواند خطرناک باشد)، اما در سیستم‌های فیلتر اسپم Precision اهمیت بیشتری دارد (نباید ایمیل‌های غیر اسپم را به اشتباه اسپم شناسایی کند).

۸. مساله Precision-Recall Tradeoff

❖ یک نفر فقط از ۵ مدل کفش خوشش می‌آید و به کفش فروشی می‌رود. فروشنده به خریدار ۱۰۰ مدل کفش پیشنهاد می‌دهد و بین این ۱۰۰ مدل، ۴ مدل از آن دسته‌ای است که خریدار خوشش می‌آید. Precision و Recall فروشنده چقدر است؟ Confusion Matrix را بسازید.

- تعداد مدل‌های مورد علاقه خریدار = ۵
- تعداد مدل‌هایی که فروشنده پیشنهاد داده = ۱۰۰
- تعداد مدل‌های درست پیش‌بینی شده (True Positives یا TP) = ۴
- تعداد مدل‌های مورد علاقه که پیشنهاد نشده‌اند (False Negatives یا FN) = ۵ - ۴ = ۱
- تعداد مدل‌های نادرست پیشنهاد شده (False Positives یا FP) = ۱۰۰ - ۴ = ۹۶

🚩 Confusion Matrix:

	پیش‌بینی مثبت (پیشنهاد شده)	پیش‌بینی منفی (پیشنهاد نشده)
واقعی مثبت	4 (TP)	1 (FN)
واقعی منفی	96 (FP)	0 (TN)

🚩 Precision و Recall:

➤ Precision:

$$Precision = \frac{TP}{TP + FP} = \frac{4}{4 + 96} = 0.04$$

➤ Recall:

$$Recall = \frac{TP}{TP + FN} = \frac{4}{4 + 1} = 0.8$$

❖ حالت دوم: فروشنده ۴ مدل پیشنهاد می‌دهد و خریدار از ۳ مدل خوشش می‌آید. در این حالت Precision و Recall چقدر است؟ برای این مثال هم Confusion Matrix را بنویسید.

- تعداد مدل‌های مورد علاقه خریدار = ۵
- تعداد مدل‌هایی که فروشنده پیشنهاد داده = ۴
- تعداد مدل‌های درست پیش‌بینی شده (TP یا True Positives) = ۳
- تعداد مدل‌های مورد علاقه که پیشنهاد نشده‌اند (FN یا False Negatives) = ۵ - ۳ = ۲
- تعداد مدل‌های نادرست پیشنهاد شده (FP یا False Positives) = ۴ - ۳ = ۱

🚦 Confusion Matrix:

	پیش‌بینی مثبت (پیشنهاد شده)	پیش‌بینی منفی (پیشنهاد نشده)
واقعی مثبت	3 (TP)	2 (FN)
واقعی منفی	1 (FP)	94 (TN)

🚦 Precision و Recall:

➤ Precision:

$$Precision = \frac{TP}{TP + FP} = \frac{3}{3 + 1} = 0.75$$

➤ Recall:

$$Recall = \frac{TP}{TP + FN} = \frac{3}{3 + 2} = 0.6$$

نتیجه‌گیری:

- در حالت اول، Recall بالا (0.8) است زیرا فروشنده اکثر مدل‌های مورد علاقه خریدار را پیشنهاد داده، اما Precision پایین (0.04) است چون مدل‌های زیادی نادرست پیشنهاد شده‌اند.
- در حالت دوم، Precision بالا (0.75) است زیرا اکثر مدل‌های پیشنهادی درست هستند، اما Recall پایین (0.6) است چون برخی از مدل‌های مورد علاقه خریدار پیشنهاد نشده‌اند.

Tradeoff

- اگر هدف ارائه مدل‌های مرتبط‌تر باشد، باید Precision را افزایش داد (مثل حالت دوم).
- اگر هدف پوشش دادن حداکثری مدل‌های مورد علاقه باشد، باید Recall را افزایش داد (مثل حالت اول).

۹. یک مدل برای تشخیص Fraud در سیستم‌های مالی توسعه داده‌ایم. نسبت حضور هر کلاهبرداری به انتقال سالم ۱ به ۱۰۰۰۰ است. دیتاست هم از همین نسبت پیروی می‌کند. توضیح دهید چگونه این سیستم را آموزش می‌دهید و ارزیابی می‌کنید؟

چگونه سیستم تشخیص کلاهبرداری را آموزش دهیم و ارزیابی کنیم؟

با توجه به نسبت حضور کلاهبرداری به انتقال سالم (۱ به ۱۰۰۰۰)، مسئله ما یک طبقه‌بندی با داده‌های نامتوازن است. برای آموزش و ارزیابی مدل، باید به طور خاص به این عدم توازن توجه کنیم.

۱. پیش‌پردازش داده‌ها:

(a) متعادل‌سازی داده‌ها (Balancing):

○ **Undersampling**: کاهش تعداد نمونه‌های کلاس غالب (انتقال سالم) به طوری که با کلاس اقلیت (کلاهبرداری) تعادل پیدا کند.

▪ مزیت: سرعت پردازش بالاتر.

▪ عیب: ممکن است اطلاعات مهم از دست برود.

○ **Oversampling**: افزایش نمونه‌های کلاس اقلیت با استفاده از تکنیک‌هایی مانند Synthetic SMOTE (Minority Over-Sampling Technique).

▪ مزیت: حفظ داده‌های کلاس غالب.

▪ عیب: احتمال Overfitting در کلاس اقلیت.

(b) ویژگی‌های متوازن:

○ حذف ویژگی‌های غیرضروری که ممکن است بایاس ایجاد کنند.

○ ایجاد ویژگی‌های جدید که بهتر بتوانند رفتار کلاهبرداری را نشان دهند.

2. انتخاب مدل:

• استفاده از مدل‌هایی که با داده‌های نامتوازن بهتر کار می‌کنند، مانند:

○ **Random Forest** (با تنظیم وزن نمونه‌ها)

○ **XGBoost** یا **LightGBM** (با تنظیم پارامترهای مرتبط با کلاس‌ها)

○ **Logistic Regression** با اضافه کردن وزن برای کلاس اقلیت.

3. ارزیابی مدل:

با توجه به عدم توازن، دقت (Accuracy) معیار مناسبی برای ارزیابی نیست. به جای آن از معیارهای زیر استفاده شود:

(a) Precision و Recall:

○ Precision: درصد کلاهبرداری‌های درست شناسایی شده از میان تمام پیش‌بینی‌های مثبت.

○ Recall: درصد کلاهبرداری‌های شناسایی شده از میان تمام کلاهبرداری‌های واقعی.

F1-Score (b)

○ میانگین موزون Precision و Recall که برای کلاس اقلیت مفید است:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

ROC-AUC (c)

○ مساحت زیر منحنی ROC، که نشان‌دهنده توانایی مدل در تمایز بین کلاهداردی و انتقال سالم است.

Precision-Recall Curve (d)

○ برای داده‌های نامتوازن مناسب‌تر از ROC-AUC است.

4. استراتژی‌های پیشرفته:

a) Threshold Tuning:

○ تنظیم کردن مقدار آستانه پیش‌بینی (Threshold) تا تعادل بهتری بین Precision و Recall برقرار شود.

b) Cost-sensitive Learning:

○ اختصاص دادن جریمه بیشتر برای خطاهای مربوط به کلاهداردی (مثلاً با تنظیم وزن در تابع هزینه).

c) Anomaly Detection:

○ اگر داده‌های کلاهداردی بسیار کم است، از روش‌های شناسایی ناهنجاری مثل Isolation Forest یا Autoencoder استفاده شود.

5. تست و مانیتورینگ:

- مدل را روی یک مجموعه داده واقعی تست کنید که نسبت کلاهداردی به انتقال سالم در آن حفظ شده باشد.
- عملکرد مدل را به طور مداوم مانیتور کنید تا در صورت تغییر الگوهای کلاهداردی، مدل به‌روزرسانی شود.

نتیجه‌گیری:

برای آموزش و ارزیابی مدل تشخیص کلاهداردی:

۱. از تکنیک‌های متوازن‌سازی داده‌ها استفاده کنید.
۲. از معیارهای مناسب مانند Precision, Recall و F1-Score برای ارزیابی استفاده کنید.
۳. مدل را با تنظیم آستانه و روش‌های حساس به هزینه بهینه کنید.
۴. عملکرد را به طور مداوم بررسی کنید تا مدل برای تغییرات آینده آماده باشد.

۱۰. مفهوم منحنی Receiver Operating Characteristic (ROC) و Area Under the Curve (AUC) را توضیح دهید.

مفهوم ROC و AUC

- Receiver Operating Characteristic (ROC):

منحنی ROC ابزاری گرافیکی برای ارزیابی عملکرد مدل‌های دسته‌بندی، به‌ویژه در مسائل دسته‌بندی باینری، است. این منحنی نشان‌دهنده رابطه بین نرخ مثبت صحیح (True Positive Rate - TPR) و نرخ مثبت کاذب (False Positive Rate - FPR) برای مقادیر مختلف آستانه (threshold) تصمیم‌گیری مدل است.

محور عمودی (Y): True Positive Rate.

$$TPR = \frac{TP}{TP + FN}$$

محور افقی (X): False Positive Rate.

$$FPR = \frac{FP}{FP + TN}$$

ROC با تغییر آستانه پیش‌بینی مدل (Threshold) رسم شده و به نوعی برای تمام آستانه‌های احتمالی (۰ تا ۱) مدل ترسیم می‌شود.

نقطه‌ای که روی محور $FPR=0$ و $TPR=1$ قرار دارد (بالا-چپ)، بهترین عملکرد را نشان می‌دهد.

- Area Under the Curve (AUC):

AUC مقدار مساحت زیر منحنی ROC را محاسبه می‌کند و معیاری عددی برای ارزیابی عملکرد مدل ارائه می‌دهد. این مقدار در بازه $[0,1]$ قرار دارد:

$AUC=1$: مدل کامل است و همه نمونه‌ها را به‌درستی دسته‌بندی می‌کند.

$AUC=0.5$: مدل کاملاً تصادفی عمل می‌کند (بدون توانایی تشخیص مثبت‌ها و منفی‌ها).

$AUC<0.5$: عملکرد مدل بدتر از تصادف است (احتمالاً برچسب‌ها معکوس شده‌اند).

AUC نشان می‌دهد که مدل با چه دقتی می‌تواند نمونه‌های مثبت را از منفی تفکیک کند. به طور خاص، بیان می‌کند که اگر یک نمونه مثبت و یک نمونه منفی انتخاب شوند، مدل چه احتمالی دارد که نمونه مثبت را به عنوان امتیاز بالاتر پیش‌بینی کند.

❖ چگونه امتیاز AUC را در مسائل طبقه‌بندی تفسیر می‌کنید؟ اگر AUC یک مدل ۰,۷ باشد در مقابل ۰,۹ چه معنایی دارد؟

مقایسه $AUC = 0.7$ و $AUC = 0.9$:

- $AUC = 0.7$:
مدل ۷۰٪ از زمان می‌تواند به درستی تشخیص دهد که یک نمونه مثبت نسبت به یک نمونه منفی امتیاز بالاتری دارد. این نشان‌دهنده عملکرد مناسب است، اما مدل هنوز بهینه نیست و ممکن است در شرایطی عملکرد نادرستی داشته باشد.
- $AUC = 0.9$:
مدل ۹۰٪ از زمان می‌تواند به درستی نمونه‌های مثبت و منفی را تشخیص دهد. این نشان‌دهنده عملکرد بسیار خوب مدل است و نشان می‌دهد که مدل توانایی بالایی در تمایز بین کلاس‌ها دارد.

تفسیر عملی:

- مدل با $AUC = 0.7$:
مناسب برای کاربردهایی است که حساسیت یا دقت مطلق نیاز نیست و خطاهای نسبتاً بیشتری پذیرفته می‌شود. (مثال: پیشنهادات اولیه برای کاربران در یک سیستم توصیه‌گر)
- مدل با $AUC = 0.9$:
مناسب برای کاربردهای حساس مانند تشخیص بیماری یا سیستم‌های امنیتی که دقت بالایی نیاز دارند.

❖ تفاوت Precision-Recall curve را با ROC بررسی کنید و مثالی بزنید که Precision برای ارزیابی مناسب‌تر باشد.

تفاوت Precision-Recall Curve با ROC

هر دو منحنی ابزارهایی برای ارزیابی عملکرد مدل‌های دسته‌بندی باینری هستند، اما تمرکز و موارد کاربرد آن‌ها متفاوت است.

➤ ROC Curve (Receiver Operating Characteristic):

- ROC رابطه بین True Positive Rate (TPR) و False Positive Rate (FPR) را برای مقادیر مختلف آستانه (threshold) نمایش می‌دهد.
- تمرکز آن بر توازن بین حساسیت (Sensitivity) و خصوصیت (Specificity) است و برای داده‌هایی که کلاس‌هایش تقریباً متعادل هستند، مناسب است.
- کاربرد آن در مشکلاتی است که اهمیت خطا در طبقه‌بندی هر دو کلاس (مثلاً مثبت و منفی) برابر است.

➤ Precision-Recall Curve:

- Precision-Recall (PR) رابطه بین Precision و Recall را نشان می‌دهد:
 - Precision: نسبت نمونه‌های مثبت درست شناسایی شده به تمام نمونه‌های پیش‌بینی شده مثبت.
 - Recall (Sensitivity): نسبت نمونه‌های مثبت درست شناسایی شده به تمام نمونه‌های مثبت واقعی.
- تمرکز آن بر عملکرد مدل در شناسایی نمونه‌های مثبت است و برای داده‌هایی با عدم توازن بالا (Imbalanced Data)، مناسب است، یعنی زمانی که کلاس مثبت بسیار کمتر از کلاس منفی است.
- کاربرد آن زمانی که تمرکز اصلی بر کاهش مثبت‌های کاذب (FP) یا بهبود دقت در پیش‌بینی نمونه‌های مثبت باشد.

مقایسه کلی:

ویژگی	ROC Curve	Precision-Recall Curve
محورهای گراف	TPR vs. FPR	Precision vs. Recall
مناسب برای	داده‌های متعادل	داده‌های نامتعادل
تمرکز	روی عملکرد کلی مدل	روی شناسایی نمونه‌های مثبت
حساس به کلاس مثبت	کمتر	بیشتر

مثالی برای استفاده از Precision-Recall Curve

فرض کنید در یک مسئله تشخیص بیماری نادر:

- داده‌ها نامتعادل هستند: فقط ۱٪ از بیماران واقعاً به بیماری مبتلا هستند (کلاس مثبت)، در حالی که ۹۹٪ به بیماری مبتلا نیستند (کلاس منفی).
- هدف: شناسایی بیماران واقعی (کاهش نمونه‌های مثبت کاذب و افزایش دقت).

چرا Precision مهم‌تر است؟

- اگر از ROC استفاده کنید، مدل ممکن است امتیاز بالایی داشته باشد زیرا کلاس منفی غالب است و نرخ مثبت کاذب پایین باقی می‌ماند.
- اما در Precision-Recall، تمرکز روی شناسایی دقیق بیماران است. Precision به شما نشان می‌دهد که پیش‌بینی‌های مثبت مدل چقدر دقیق هستند و چند درصد از آن‌ها واقعاً مثبت هستند.

جمع‌بندی:

- ROC Curve برای داده‌های متعادل و ارزیابی کلی مناسب است.
- Precision-Recall Curve زمانی اهمیت دارد که داده‌ها نامتعادل باشند یا طبقه مثبت دارای اهمیت بالایی باشد.
- در مثال تشخیص بیماری نادر، Precision-Recall Curve گزینه بهتری برای ارزیابی عملکرد مدل است.

۱۱. فرآیند K-fold cross-validation را توضیح دهید و چگونه برای ارزیابی عملکرد مدل استفاده می‌شود. مزایا و معایب K-fold cross-validation در مقایسه با تقسیم ساده داده‌ها به مجموعه آموزش و آزمایش چیست؟ چگونه مقدار k را انتخاب می‌کنید؟

فرآیند K-fold Cross-Validation

K-fold cross-validation یک روش اعتبارسنجی برای ارزیابی عملکرد مدل یادگیری ماشین است که داده‌ها را به صورت هوشمندانه به مجموعه‌های آموزشی و آزمون تقسیم می‌کند تا نتایج بهتری حاصل شود.

مراحل اجرای K -fold Cross-Validation:

۱. تقسیم داده‌ها:
 - داده‌ها به K بخش (Fold) با اندازه‌های برابر تقسیم می‌شوند.
۲. آموزش و ارزیابی:
 - مدل روی $K-1$ بخش آموزش داده می‌شود و روی بخش باقی‌مانده تست می‌شود.
 - این فرآیند K بار تکرار می‌شود، به طوری که هر بخش یک بار به عنوان مجموعه تست عمل می‌کند.
۳. محاسبه عملکرد:
 - معیارهای ارزیابی (مثل دقت، F1-Score یا MSE) در هر یک از K اجرا محاسبه می‌شوند.
 - میانگین این معیارها به عنوان عملکرد نهایی مدل گزارش می‌شود.

مزایا و معایب K -fold cross-validation در مقایسه با تقسیم ساده داده‌ها به مجموعه آموزش و آزمایش

○ مزایای K -Fold Cross-Validation:

۱. استفاده بهینه از داده‌ها:
 - در K -Fold Cross-Validation، همه داده‌ها به‌طور متناوب در مجموعه‌های آموزش و آزمایش قرار می‌گیرند. بنابراین، از تمام داده‌ها برای آموزش و آزمایش استفاده می‌شود و هیچ بخشی از داده‌ها به‌طور کامل کنار گذاشته نمی‌شود.
 - در مقابل، در تقسیم ساده داده‌ها، بخشی از داده‌ها فقط برای آزمایش استفاده می‌شوند و در آموزش مدل نقشی ندارند.
۲. کاهش واریانس نتایج:
 - در K -Fold، نتایج حاصل از چندین ترکیب از داده‌ها میانگین‌گیری می‌شوند، که ارزیابی پایدارتری از عملکرد مدل ارائه می‌دهد.
 - در تقسیم ساده، نتایج ممکن است وابسته به نحوه انتخاب تصادفی مجموعه‌های آموزش و آزمایش باشند و این می‌تواند منجر به ناپایداری شود.
۳. کاهش بیش‌برازش (Overfitting):
 - چون مدل چندین بار بر روی داده‌های مختلف آزمایش می‌شود، احتمال اینکه مدل بیش‌برازش کند کاهش می‌یابد.
 - در تقسیم ساده، ارزیابی مدل فقط بر اساس یک مجموعه آزمایشی انجام می‌شود که ممکن است نماینده خوبی از داده‌ها نباشد.
۴. قابلیت ارزیابی مدل در داده‌های متنوع‌تر:
 - هر نمونه داده‌ای در هر Fold حداقل یک بار به‌عنوان داده آزمایشی استفاده می‌شود، که تنوع بیشتری در ارزیابی فراهم می‌کند.

○ معایب K -Fold Cross-Validation:

۱. هزینه محاسباتی بالا:
 - K -Fold به K بار آموزش و آزمایش مدل نیاز دارد. این فرآیند، به‌ویژه برای مدل‌های پیچیده یا داده‌های بزرگ، بسیار زمان‌بر و محاسباتی سنگین است.
 - در تقسیم ساده، مدل فقط یک بار آموزش داده و آزمایش می‌شود، که سریع‌تر است.
۲. پیچیدگی پیاده‌سازی:

- K-Fold پیچیده‌تر از تقسیم ساده داده‌ها است و به پیاده‌سازی دقیق نیاز دارد.
- در مقابل، تقسیم ساده مستقیم و ساده است.
- ۳. عدم تعادل داده‌ها در برخی Folds:
 - اگر داده‌ها نامتعادل باشند (مثلاً تعداد نمونه‌های هر کلاس متفاوت باشد)، ممکن است برخی از Folds شامل نمونه‌های کافی از یک کلاس نباشند و ارزیابی مدل دچار bias شود. هرچند می‌توان از **Stratified K-Fold** برای کاهش این مشکل استفاده کرد.
- ۴. نیاز به انتخاب پارامتر K:
 - انتخاب مقدار مناسب K می‌تواند چالش‌برانگیز باشد. مقدار کوچک (مثلاً $K=2$) ممکن است ارزیابی مناسبی ارائه ندهد و مقدار بزرگ‌تر (مثلاً $K=10$) محاسبات را افزایش می‌دهد.

○ مزایای تقسیم ساده داده‌ها:

۱. سرعت و سادگی:
 - تقسیم ساده بسیار سریع است و به پیاده‌سازی پیچیده نیاز ندارد.
۲. عملکرد قابل قبول در داده‌های بزرگ:
 - در داده‌های بزرگ، تقسیم ساده بعضاً می‌تواند به اندازه کافی نماینده باشد و نیازی به روش‌های پیچیده‌تر مانند K-Fold نباشد.

○ معایب تقسیم ساده داده‌ها:

۱. واریانس بالا:
 - نتایج مدل ممکن است به نحوه تقسیم تصادفی داده‌ها وابسته باشد و عملکرد مدل ناپایدار باشد.
۲. استفاده ناکارآمد از داده‌ها:
 - بخشی از داده‌ها (داده‌های آزمایشی) در آموزش استفاده نمی‌شوند و این ممکن است در داده‌های کوچک باعث کاهش عملکرد مدل شود.
۳. احتمال عدم تعادل:
 - ممکن است نمونه‌های خاصی به‌طور تصادفی به یکی از مجموعه‌ها تخصیص یابند که نتایج را تحت تأثیر قرار می‌دهد.

جمع‌بندی:

○ **K-Fold Cross-Validation** مناسب‌تر است زمانی که:

- داده‌ها محدود یا نامتعادل هستند.
- ارزیابی پایدارتر و دقیق‌تری نیاز است.
- درگیر Overfitting هستیم.
- هزینه محاسباتی مشکلی ایجاد نمی‌کند.

○ **تقسیم ساده داده‌ها** مناسب‌تر است زمانی که:

- داده‌ها بزرگ هستند و نمایندگی کافی از کل مجموعه دارند.
- نیاز به سرعت و سادگی بیشتری در فرآیند آزمایش و آموزش دارید.

انتخاب مقدار K :

- مقادیر رایج:
 - $K=5$ یا $K=10$ به طور معمول استفاده می‌شود. این مقادیر تعادل خوبی بین دقت و هزینه محاسباتی ایجاد می‌کنند.
- اندازه داده‌ها:
 - اگر داده‌ها کم باشند، K بزرگ‌تر انتخاب می‌شود تا از داده‌های بیشتری برای آموزش استفاده شود.
 - اگر داده‌ها زیاد باشند، $K=5$ کافی است.
- Leave-One-Out Cross-Validation (LOOCV):
 - حالتی خاص که K برابر تعداد نمونه‌هاست (هر نمونه یک بار به عنوان تست استفاده می‌شود).
 - هزینه محاسباتی بالا دارد و فقط برای داده‌های بسیار کوچک مناسب است.

۱۲. فرض کنید به خواسته **saman-kala.ir** یک سیستم پیشنهاد دهنده کالا برای مشتریان طراحی کرده‌اید. در کمال تعجب سیستم پیشنهاد دهنده قبلی، به صورت دستی توسط کارمندان انجام می‌شد!

❖ حال مدیر پروژه از شما می‌خواهد پس از اجرای پروژه پیشنهاد خودتان، به آن‌ها نشان دهید مدل شما از مدل قبلی بهتر است. (راهنمایی: برای اینکه بگوییم یک الگوریتم بهتر است باید یک (یا چند) معیار داشته باشیم که بتوانیم مقایسه کنیم. شما به عنوان دیتاساینتیست باید در تعریف این معیارها کمک کنید.)

اثبات برتری مدل پیشنهادی نسبت به مدل قبلی:

برای اینکه نشان دهیم سیستم پیشنهادی ما بهتر از روش دستی قبلی است، باید از **معیارهای کمی و کیفی** استفاده کنیم:

○ معیارهای کمی (Quantitative Metrics):

۱. دقت پیشنهادها (Accuracy):
 - بررسی درصد پیشنهادهایی که توسط مشتری انتخاب می‌شوند.
 - مثال: نرخ کلیک (CTR - Click Through Rate) یا نرخ خرید از پیشنهادها (Conversion Rate).
۲. تنوع پیشنهادها (Diversity):
 - اندازه‌گیری تنوع کالاهای پیشنهاد شده به مشتریان. سیستم دستی ممکن است پیشنهادات تکراری ارائه دهد.
۳. نرخ نگهداری مشتریان (Retention Rate):
 - بررسی تأثیر سیستم پیشنهادی بر نرخ بازگشت مشتریان.
۴. سرعت پاسخ‌دهی:
 - زمان لازم برای تولید پیشنهادات توسط سیستم در مقایسه با روش دستی.

○ معیارهای کیفی (Qualitative Metrics):

۱. تجربه مشتری (Customer Satisfaction):
 - نظرسنجی از مشتریان درباره رضایت از پیشنهادها ارائه شده.
۲. سهولت استفاده:
 - بررسی نظر کارمندان و مشتریان درباره راحتی استفاده از سیستم جدید.

روش کار:

- **آزمایش A/B Testing:** دو گروه مشتری ایجاد می‌کنیم، یک گروه با سیستم قبلی (روش دستی) و گروه دیگر با سیستم پیشنهادی کار کنند. سپس معیارهای فوق را برای هر دو گروه مقایسه می‌کنیم.

❖ همچنین مدیر سایت، از تیم دیتا تخمین زمان می‌خواهد که هزینه این پروژه چه مدت پس از اجرا شدن آن، توسط سودش برمی‌گردد (ROI). پس باید روشی را پیشنهاد دهید که مشخص کند این پروژه چقدر برای شرکت سودآوری دارد. (راه دستیابی به این جواب‌ها را با ذکر فرض‌های منطقی خودتان، توضیح دهید).

تخمین بازگشت سرمایه (ROI):

برای تخمین زمان بازگشت هزینه پروژه، نیاز به ارزیابی موارد زیر داریم:

فرضیات:

۱. هزینه اجرای پروژه (Cost of Implementation):
 - شامل هزینه توسعه سیستم، نگهداری، و آموزش کارکنان.
۲. درآمد حاصل از سیستم جدید (Incremental Revenue):
 - افزایش درآمد ناشی از بهبود تجربه مشتری، افزایش فروش و نگهداری مشتریان.

محاسبه ROI:

۱. فرمول ROI:

$$ROI = \frac{\text{هزینه کل} - \text{درآمد اضافی حاصل از سیستم}}{\text{هزینه کل}}$$

۲. برآورد زمان بازگشت هزینه:
 - نقطه زمانی را پیدا کنید که درآمد اضافی سیستم با هزینه پروژه برابر شود.

روش دستیابی:

۱. تحلیل درآمد گذشته:
 - بررسی فروش‌های حاصل از سیستم قبلی جهت تخمین پایه‌ای برای درآمد.
۲. پیش‌بینی سود سیستم جدید:
 - استفاده از مدل‌های پیش‌بینی مالی (مانند مدل‌های رگرسیون یا سری‌های زمانی) برای تخمین تأثیر سیستم جدید.
۳. شبیه‌سازی:
 - استفاده از تکنیک‌های شبیه‌سازی برای ارزیابی زمان‌بندی بازگشت سرمایه در شرایط مختلف.